

CCTV Abnormal Behavior Detection

Full Plan: NTU RGB+D Training Data → MediaPipe Live Deployment

CSE-445 Project

Target Classes

- **Normal Activity** — mapped from NTU codes A001, A008, A009
- **Staggering / Unbalanced Movement** — A042
- **Falling Down** — A043
- **Touch Chest** — A045
- **Motionless After Fall** — rule-based post-processing state, not a trained class

Project Scope

Per supervisor's instruction, the system detects only the 4 behavior classes above. It is a behavioral-alert system, not a medical diagnosis system — it does not detect heart attacks or other medical conditions directly, only the visible movement patterns that may be associated with them.

1 Full Pipeline, Start to Finish

1.1 Step 1 — Prepare the raw NTU RGB+D data

Obtain the NTU RGB+D dataset and isolate only the files needed for the 6 target action codes (A001, A008, A009, A042, A043, A045). Use the official NTU missing-skeleton list to identify and exclude corrupted files before any processing begins.

1.2 Step 2 — Parse raw skeleton files into a clean frame-level table

Write a script that reads every `.skeleton` file and produces one combined CSV. For each frame: keep only x/y/z coordinates per joint (discard depth-pixel, color-pixel, quaternion, and tracking-state columns), drop frames where a joint was not reliably tracked, and tag every row with file metadata (subject ID, action code) and the mapped class label. This is the foundational raw dataset everything else builds on.

1.3 Step 3 — Build a metadata manifest (recommended before windowing)

Separately, scan every file at a structural level only — frame count, body count, percentage of frames with tracking issues — without extracting full coordinates. This manifest is used purely for planning: it reveals real clip-length distributions per class and confirms the true scale of class imbalance in terms of frame count, not just file count.

1.4 Step 4 — Recompute NTU’s spine joints the “MediaPipe way”

NTU’s Kinect sensor provides 4 real spine measurements: SpineBase, SpineMid, Neck, SpineShoulder. MediaPipe cannot supply these directly — it can only approximate them as midpoints (e.g. SpineBase $\approx$ midpoint of left/right hip). To avoid a training/inference mismatch, **discard NTU’s native spine values and recompute all 4 using the same midpoint formulas MediaPipe will use live**. This keeps training data and live data built by the same process.

1.5 Step 5 — Select the canonical joint set

Keep only the joints both NTU and MediaPipe can supply: recomputed spine points, shoulders, elbows, wrists, hips, knees, ankles, and head. Drop fine hand/finger joints (HandTip, Thumb) — not required for these 4 classes, and noisy in MediaPipe.

1.6 Step 6 — Normalize consistently

For every frame, subtract the (recomputed) SpineBase position from every joint, and scale by a body-size reference such as shoulder-to-hip distance. This removes distance-from-camera and subject-size effects. The exact same normalization function must be applied to NTU data and MediaPipe data later — not just the same idea, the same code.

1.7 Step 7 — Window the normalized frames

Slice each continuous sequence into fixed-length clips (e.g. 30 or 60 frames, with a chosen stride/overlap). This turns variable-length sequences into fixed-size samples a model can consume.

1.8 Step 8 — Extract features per window

For each window, compute the signals the model will actually learn from:

- Joint velocity (mean/max speed) — primary signal for Falling Down and Staggering
- Vertical drop of SpineBase/Head over the window — primary Falling Down signal
- Torso tilt angle (SpineBase→Neck vector vs. vertical) — primary Staggering signal
- Minimum hand-to-chest distance — primary Touch Chest signal
- Position spread (mean/std) per joint — general posture signal

1.9 Step 9 — Write ONE shared feature function, with two adapters

Implement a single `compute_features(joint_dict)` function that takes a plain dictionary of canonical joint coordinates and returns the feature row. Then write two small adapter functions: one converts an NTU frame into that dictionary format, the other converts a MediaPipe frame into the same format. Both adapters feed the same core function, guaranteeing identical math everywhere — now and later, in the live system.

1.10 Step 10 — Build the labeled training table

One row per window: extracted features + class label. This is the actual input to the classifier.

1.11 Step 11 — Split by subject, not randomly

Split train/validation/test by subject ID so the same person’s movement never appears in more than one split. Random row-wise splitting leaks information and inflates accuracy.

1.12 Step 12 — Handle class imbalance at training time

Normal Activity will produce roughly $3\times$ more windows than each abnormal class. Apply class weighting or balanced sampling during training — do not bake balancing into the stored dataset file itself, so it stays easy to experiment with.

1.13 Step 13 — Train the model

Fit a classifier (Random Forest, XGBoost, or a small feedforward network are all reasonable for this data size) on the labeled training table.

1.14 Step 14 — Build the live MediaPipe pipeline

Camera $\rightarrow$ MediaPipe pose landmarks $\rightarrow$ MediaPipe adapter (Step 9) $\rightarrow$ same normalization (Step 6) $\rightarrow$ same feature function (Step 9) $\rightarrow$ trained model $\rightarrow$ prediction. Reusing the exact same code as training guarantees the live system matches what the model actually learned.

1.15 Step 15 — Validate with real MediaPipe footage before trusting it

Record a handful of real clips per class using the actual live pipeline (not NTU data), run them through the trained model, and sanity-check the predictions. This is the only step that can catch a subtle mismatch that NTU-only validation cannot reveal.

2 Why the Order Matters

Steps 4, 6, and 9 exist specifically to prevent a single, easy-to-miss failure: **training the model on real Kinect measurements while the live system only ever sees MediaPipe approximations**. If those two data sources are built differently, the model learns patterns that will not transfer to the live camera feed — even if accuracy on NTU alone looks excellent. Building one shared feature function with two adapters (Step 9) is the safeguard that keeps this from happening as the project grows.